

TECHNICAL DOCUMENTATION

React & Vite Portfolio Application Architecture

AUTHOR:

Viketh Hegde

PROJECT:

Personal Developer Portfolio SPA

DATE / VERSION:

August 2026 / Version 1.0.0

1. Technical Stack Overview

The portfolio website is a modern Single Page Application (SPA) designed to showcase software engineering projects, skills, and academic milestones. It balances high-contrast minimalist aesthetics with responsive layout systems, smooth physics scrolling, and interactive micro-animations.

* Core Framework: React 19

Manages the DOM lifecycle, state dependencies, interactive drawers, and components. Leverages React's virtual DOM structure for fast, client-side re-renders.

* Build Tool & Bundler: Vite 8

Serves as the local development compiler and production bundler. Utilizes lightning-fast hot module replacement (HMR) and optimized Rollup configurations to deliver highly compressed client assets (JS, CSS, and images).

* Styling System: Tailwind CSS v4

Applies responsive utility styles dynamically. Uses a modern CSS-in-JS utility-first styling pipeline. Registers custom typography variables inside the native CSS theme configuration.

* Inertial Smooth Scrolling: Lenis Scroll Engine

Implements smooth inertia physics calculations for scroll animations. Lenis binds browser scroll events, applying fluid friction curves for smooth wheel and touchpad gliding.

* Iconography Library: Lucide React

Provides optimized SVG stroke graphics loaded dynamically as modular React components.

* Form Submission Pipeline: Formspree API

Processes contact form messages asynchronously using Fetch API AJAX requests, removing the need for dedicated server hosting while managing submit success, error, and loader state transitions.

2. Directory Structure

Below is the structure of the project directory, delineating configuration scripts, static public assets, and components in the src/ folder:

```
Portfolio/
|-- index.html           # Main SPA HTML container
|-- package.json         # Project dependencies & npm scripts
|-- vite.config.js       # Vite environment bundler configurations
|-- public/             # Static client assets (PDF, jpeg files)
|   |-- resume.pdf       # Downloadable curriculum vitae
|   |-- image.jpeg       # Portrait thumbnail used in contact card
|   \-- favicon.svg      # Tab icon metadata
\-- src/                 # React source code root
    |-- main.jsx         # Client entry script
    |-- App.jsx          # Root shell and layout orchestrator
    |-- index.css        # Global styles, variables, & keyframes
    |-- fonts.css        # Embedded custom typography files
    \-- components/      # Shared component folder
        |-- Hero.jsx     # Key typography layout
        |-- About.jsx    # Bio and resume triggers
        |-- Skills.jsx   # Animated tech stack block
        |-- Journey.jsx  # Sticky history timeline list
        |-- Projects.jsx # Code repos grids and details
        |-- Contact.jsx  # Integrated Formspree mail block
        |-- Menu.jsx     # Nav overlays and scrolling triggers
        |-- Assistant.jsx # Float chat panel drawer
        |-- CustomCursor.jsx # Position coordinate inverting dot
        |-- Preloader.jsx # Typographic preloading landing screen
        |-- ScrollReveal.jsx # Observer layout wrapper
        |-- InteractiveString.jsx # Canvas dynamic elastic string line
        |-- Ticker.jsx   # Infinite horizontal loop marquee
        \-- ScrambleText.jsx # Alphanumeric text scramble helper
```

3. Detailed File Breakdown

* **index.html**

The root HTML5 shell template. Hooks in the bundle entry point `src/main.jsx` and defines browser title/meta headers.

* **src/main.jsx**

The entry script that binds the React virtual DOM tree directly to the HTML document root node.

* **src/App.jsx**

The primary controller component. Sets up global menu toggle and load states, coordinates Lenis scrolling in a lifecycle loop, mounts cursor and overlay systems, and structures all content sections in a clean stack.

* **src/index.css**

Imports Tailwind CSS. Establishes core color variables, custom scrollbar styles, Infinite Marquee keyframes, and menu scale/translate transitions.

* **src/fonts.css**

Registers local typography font faces, providing support for specialized header fonts like Akira and Clash Display.

* **src/components/Hero.jsx**

The first fold header component. Uses high-contrast styling and bold display type to create a strong first impression.

* **src/components/About.jsx**

The bio layout. Houses external social profile links (GitHub, LinkedIn) and links that prompt downloading or viewing the local resume PDF file.

* **src/components/Skills.jsx**

Visualizes the tech stack in a responsive grid. Categorizes items into Languages, Frameworks, Data & DevOps, and AI/ML. Features card hover scaling, visual segmented progress meters, and dynamic description drawers. The AI/ML box displays a styled dashed-border placeholder with a pulsing animation to indicate 'in progress' studies.

* **src/components/Journey.jsx**

Implements a sticky horizontal grid layout detailing education and experience milestones. Hovering cards triggers modern shadow and vertical coordinate translations.

* **src/components/Projects.jsx**

Shows project cards containing details, used language tags, live site views, and repository buttons.

* **src/components/Contact.jsx**

Handles user emails using Fetch requests to Formspree. Shows success/loading states, maps form input values, and displays contact details.

* **src/components/Menu.jsx**

Renders the responsive full-screen link menu. Clicking options targets anchors using Lenis's smooth scrolling instance.

3. Detailed File Breakdown (Continued)

* **src/components/Assistant.jsx**

The mock AI assistant chat drawer. Installs a sliding dialog box overlay. Intercepts document scroll states and responds to user clicks with pre-programmed dialogue text typing animations.

* **src/components/CustomCursor.jsx**

Sets up a follower circle tracking cursor positions. Uses custom CSS filters (mix-blend-mode: difference) to invert background colors as it moves over light or dark elements.

* **src/components/Preloader.jsx**

Displays a landing splash loading screen with text-reveal animations. Signals complete status to release scrolling locks once components finish rendering.

* **src/components/ScrollReveal.jsx**

A scroll watcher wrapper. Utilizes an Intersection Observer hook to apply scale-up, translation, and opacity changes when layouts cross into viewport view ranges.

* **src/components/InteractiveString.jsx**

Renders canvas-style responsive lines. Leverages SVG path coordinates and page cursor vectors to bend divider strings dynamically, releasing them with a spring physics animation.

* **src/components/Ticker.jsx**

An infinite text loop component. Duplicates tag arrays to fill viewport width, executing continuous marquee keyframe slide transitions.

* **src/components/ScrambleText.jsx**

Renders a high-tech alphanumeric character cycling layout that resolves into target strings to emphasize section headers.

4. System Workflow & Integration

The components are highly integrated, responding to shared lifecycles and user gestures. The standard runtime workflow is sequenced as follows:

1. Initialization & Mounting

The browser loads `index.html`, fetches `src/main.jsx`, and compiles `src/App.jsx`. The root component boots up and initializes the global Lenis smooth scrolling engine. Global state tracking variables are created (`menuOpen`, `isLoading`).

2. The Preloading Stage

The Preloader displays immediately, presenting typographic animations while blocking scroll gestures on Lenis to prevent users from scrolling premature pages. Once asset checks resolve, the loader signals complete, updates states, and releases Lenis scrolling locks.

3. Visual Rendering & Cursor Interaction

The page renders the CustomCursor dot and Hero folding. CustomCursor tracks client mouse movements using page vector state coordinates, applying overlay blend mode filters to maintain black/white contrast on high-brightness borders.

4. Sequential Page Scrolling & Scroll Reveals

As users scroll, Lenis applies smooth friction drag curves. Individual sections (Hero, About, Skills, Journey, Projects, Contact, Footer) are wrapped in ScrollReveal. Once scroll bounds intersect the observer viewport, elements fade and translate upward in synchronization.

5. Navigation, Dialogs, & Forms

Clicking flotation menu triggers toggles the Menu full-screen overlay. Nav items glide to page targets by calling Lenis `scrollTo` commands. Opening the Assistant slides up the AI chat and stops Lenis wheel physics to allow panel focus. Writing details into the Contact form inputs and clicking submit halts input changes and fires asynchronous POST fetch queries to Formspree, updating UI success panels.